

Trabalho Prático Integração de Sistemas Informáticos

Análise de Consumo Energético de Iluminação

DOCUMENTAÇÃO TÉCNICA E GUIA DE UTILIZAÇÃO

José António da Cunha Alves (nº27967)

Instituto Politécnico do Cávado e do Ave

18 de outubro de 2025

Índice

1	Introdução e Glossário Técnico	3
1.1	Contexto Técnico de Iluminação	3
1.1.1	Equipamentos de Iluminação	3
1.2	Dados de Origem e Fluxo de Processamento	4
2	Pré-requisitos e Instalação de Dependências	4
2.1	1. Instalação de Bibliotecas Python	4
2.2	2. Instalação de Palettes Node-RED	5
3	Fluxo do Ficheiro Job Principal (a27967.kjb)	5
4	Descrição Detalhada das Transformações	6
4.1	1. Get_CueData_From_XML.ktr (Extração ds Dados exporta- dos pela grandMA2)	6
4.2	2. Patch_Import.ktr (Importação de Patch e Potência) . . .	8
4.3	3. Calculate_Power_Consumption.ktr (Cálculo de Consumo)	9
4.4	4. Graph_Creation.ktr (Geração de Gráficos)	10
4.4.1	Configuração de Caminho Absoluto do Script Python	10
4.5	5. SendEmail.ktr (Envio de Relatório)	11
5	Fluxo de Visualização Node-RED	11
5.1	Configuração de Caminhos e Ficheiros	11
5.2	Visão Geral do Fluxo Node-RED	12
5.3	Esquema de Agregação de Dados (Node-RED)	13
5.4	Componentes de Visualização (Dashboard)	13
6	Conclusão e Trabalhos Futuros	13
6.1	Limitações e Melhorias Futuras	14
7	Repositório Git	14
8	Vídeo de demonstração	14

Lista de Figuras

1	Fluxo de controlo principal definido no Job (a27967.kjb), ilustrando a sequência completa de ETL e Geração do Relatório, incluindo o <i>trigger</i> HTTP e a espera pelo Node-RED.	6
2	Esquema lógico da transformação de extração de dados da grandMA2 e anotação de tempo simulado entre <i>cues</i>	7
3	Esquema lógico da importação e separação de dados de Patch e Potência para Fixtures e Dimmers, utilizando <i>Merge Join</i>	8
4	Esquema da transformação de envio de e-mail, que recolhe os gráficos e os distribui de forma segura.	11
5	QR Code para vídeo de demonstração.	15

1 Introdução e Glossário Técnico

O presente relatório técnico descreve o fluxo de trabalho de **Extração, Transformação e Carregamento (ETL)** implementado no **Pentaho Data Integration (PDI)** (Kettle). O objetivo principal é calcular e analisar o consumo de energia ao longo de um espetáculo de teatro, a partir de dados programados numa mesa de controlo de iluminação.

1.1 Contexto Técnico de Iluminação

Este projeto utiliza conceitos fundamentais da programação de luz para teatro:

- **grandMA2:** É um sistema de controlo de iluminação profissional (mesa de luz). Permite ao designer programar e guardar o estado da luz no espetáculo.
- **Cue (Deixa):** É um estado de luz guardado. Uma *Sequence* é uma lista ordenada de *Cues* que define o espetáculo. O sistema de ETL calcula o consumo de energia durante o tempo de duração de cada *Cue*.

1.1.1 Equipamentos de Iluminação

Os projetores são classificados em duas categorias principais, essenciais para o cálculo do consumo:

- **Dimmers / Projetores Convencionais:** Equipamentos de luz simples (e.g., lâmpadas de halogéneo) cuja intensidade é controlada diretamente através de um **dimmer** (variador de intensidade). São controlados por um único valor de intensidade (0% a 100%) e têm uma potência fixa.
- **Fixtures (Projetores Inteligentes):** Equipamentos de luz modernos (e.g., *Moving Lights* ou *LEDs*). Estes são controlados por múltiplos parâmetros (cor, posição, zoom, etc) e têm um consumo de potência que pode variar consoante a função ativa. Nesta primeira fase do projeto, vamos considerar apenas o consumo do atributo de *dimmer*, ou seja, a intensidade da luz.

1.2 Dados de Origem e Fluxo de Processamento

O Kettle recebe dois tipos de ficheiros exportados da grandMA2:

- **Cue Data (XML):** Contém a intensidade de controlo (valor DMX, 0 a 255) para cada aparelho de luz (todos os seus atributos) em cada *Cue*, juntamente com os tempos de transição.
- **Patch Data (CSV):** Ficheiros de *patch* exportados da grandMA2 que contêm a informação de mapeamento de cada aparelho: ID (*Fixture/Channel*), *FixtureType*, nome e a que *Layer* (camada de iluminação) pertence. **Não contêm dados de potência.**
- **Power Consumption Data** (`power_consumption.csv`): Ficheiro criado externamente que contém a informação da **Potência Máxima** em Watts associada a cada *FixtureType* ou *Channel ID* (para Dimmers), permitindo realizar o cálculo de consumo.

O fluxo de ETL processa esta informação em cadeia:

1. Combina *Cue Data* com *Patch Data* e *Max Power*.
2. Calcula a Potência Instantânea (P_{inst}) e a Energia (Wh) consumida em cada momento do espetáculo.
3. Os resultados são usados para gerar gráficos (via Python/Node-RED) e distribuídos por e-mail.

2 Pré-requisitos e Instalação de Dependências

Para que o projeto funcione corretamente, é necessário configurar os ambientes Python (para geração de gráficos) e Node-RED (para visualização dos resultados).

2.1 1. Instalação de Bibliotecas Python

A transformação `Graph_Creation.ktr` executa um script Python (`graph_generator.py`). Este script requer bibliotecas específicas para manipulação e visualização de dados.

- **Dependências:** As bibliotecas necessárias estão no ficheiro `requirements.txt` (Pandas e Matplotlib).

- **Comando de Instalação:** Utilize o pip para instalar as dependências.

Comando a executar no terminal:

```
pip install -r requirements.txt
```

O conteúdo do ficheiro `requirements.txt` é:

```
pandas>=2.0  
matplotlib>=3.7
```

2.2 2. Instalação de Palettes Node-RED

O Node-RED é utilizado para criar **dashboards** de visualização dos dados de consumo calculados pelo Kettle. É necessário instalar as seguintes **palettes** na interface de gestão do Node-RED:

- **Dashboard (Base):**
 - node-red-dashboard
 - @flowfuse/node-red-dashboard
- **Visualização de Dados (Tabela):**
 - node-red-node-ui-table

3 Fluxo do Ficheiro Job Principal (a27967.kjb)

O ficheiro `a27967.kjb` define a sequência lógica do processamento, garantindo que as transformações sejam executadas na ordem correta, dependendo umas das outras.

O fluxo garante que:

- Os dados de *patch* e potência são importados.
- Os dados das *cues* (intensidades e tempos) são extraídos.
- O cálculo de consumo é realizado.
- Após o cálculo, o Node-RED é ativado para processar os dados de saída e gerar os ficheiros *JSON*, necessários ao script *Python*.

- O Kettle aguarda (Wait for JSON) que o Node-RED complete a geração dos ficheiros JSON de dados (pelo menos o `layer-graph-data.json`) antes de prosseguir.
- Os gráficos são gerados e enviados por e-mail.

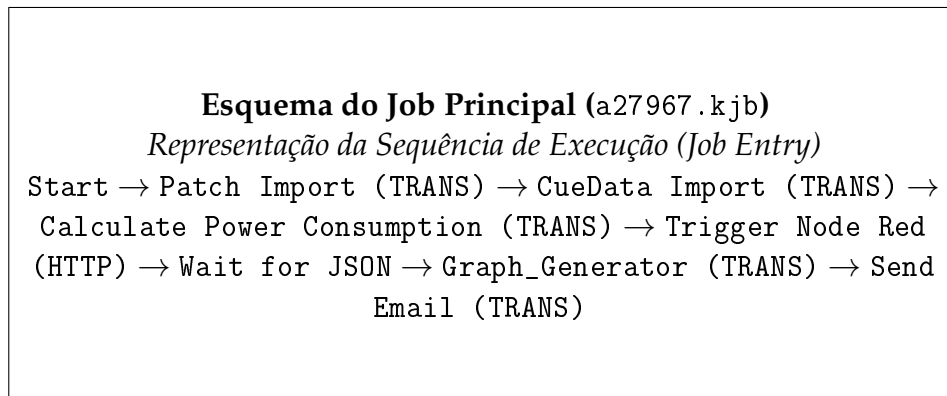


Figura 1: Fluxo de controlo principal definido no Job (a27967.kjb), ilustrando a sequência completa de ETL e Geração do Relatório, incluindo o *trigger* HTTP e a espera pelo Node-RED.

4 Descrição Detalhada das Transformações

4.1 1. Get_CueData_From_XML.ktr (Extração ds Dados exportados pela grandMA2)

Esta transformação é responsável por extrair os níveis de intensidade (DIM) e os tempos de cada *cue* a partir do ficheiro XML exportado da grandMA2 (`marat-sade.xml`). Nesta fase, ainda desprezamos os tempos de *fade* de cada *cue*, assumindo que os valores são o máximo ao longo de toda a *cue*.

- **Passo Chave:** *Get data from XML*.
- **Lógica:** A transformação itera sobre os elementos XML que contêm a informação dos canais/fixtures por *cue*.
- **Extração de Tempo:** Para simular o tempo entre *cues*, o passo *Modified JavaScript value* gera um valor aleatório (entre 10 e 420 segundos), que é associado a cada *CueNumber* e usado para calcular a energia.

O snippet do JavaScript para simular o tempo entre *cues* é:

```
// Gera um valor aleatório (em segundos) para simular o tempo entre Cues  
var betweenCuesTime = Math.floor(Math.random() * (420 - 10 + 1)) + 10;
```

Esquema da Transformação Get_CueData_From_XML.ktr

Fluxo de Extração XML e Atribuição de Tempo Simulada

Get data from XML → Remove Unnecessary Fields → Filter
rows (DIM only) → Stream lookup (Cue Times) → Export Cue
Data

Get data from XML → Select Unique Cue Numbers → Modified
JavaScript value (Random Time) → Select CueNumber and
Times → Stream lookup

Figura 2: Esquema lógico da transformação de extração de dados da grandMA2 e anotação de tempo simulado entre *cues*.

4.2 2. Patch_Import.ktr (Importação de Patch e Potência)

Esta transformação prepara os dados de *patch* e associa a potência máxima de cada equipamento.

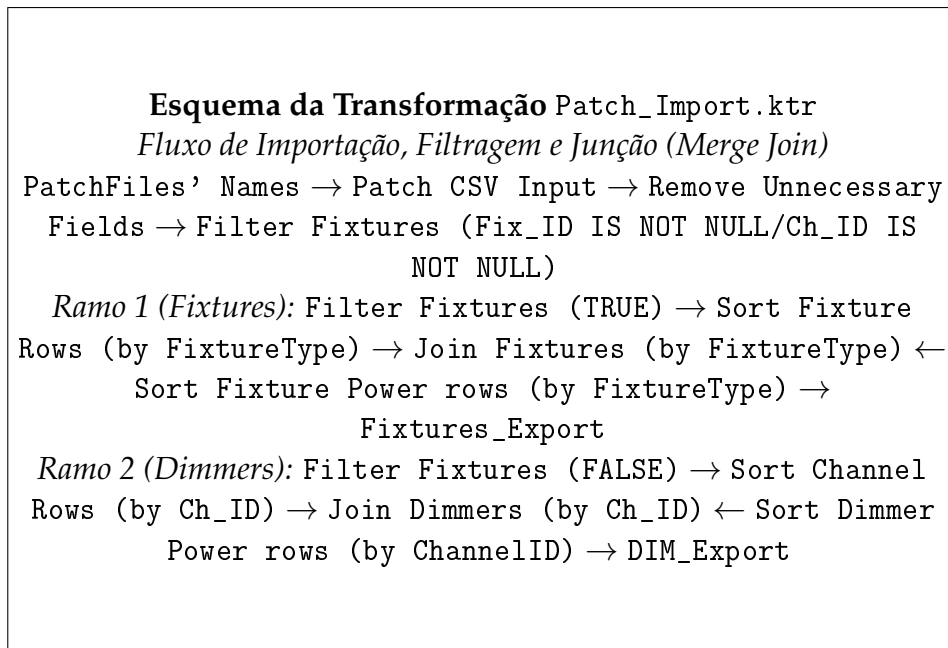


Figura 3: Esquema lógico da importação e separação de dados de Patch e Potência para Fixtures e Dimmers, utilizando *Merge Join*.

- **Origem dos Dados:** Lê o `patchfiles.csv` para obter a lista de ficheiros de patch (e.g., `ALCOVAS_LAT_t.csv`) e o `power_consumption.csv` para obter os valores de *MaxPower*.
- **Divisão e Junção:** As linhas de *patch* são divididas em Fixtures (com `Fix_ID`) e Dimmers (com `Ch_ID`). Em seguida, são utilizados passos *Merge Join* para associar o *MaxPower* (do ficheiro de consumo) por *FixtureType* ou *ChannelID*, consoante o tipo de equipamento.
- **Resultados:** Gera `fixtures_export.txt` e `dim_export.txt`.

4.3 3. Calculate_Power_Consumption.ktr (Cálculo de Consumo)

Esta é a transformação de cálculo central.

- **Entrada:** Recebe o *CueData* (intensidade DMX e tempo entre *cues*) e os dados de Patch/Potência (*MaxPower, Layer*) gerados na transformação anterior.
- **Passo Chave:** *Stream Lookup* e *Modified JavaScript value*.
- **Lógica de Lookup:** Associa o *MaxPower* e o *Layer* a cada linha de *CueData* (*Stream Lookup*, usando *FixtureID* ou *ChannelID*).
- **Lógica de Cálculo:** O passo *Modified JavaScript value* executa o cálculo de Potência e Energia:

Snippet do JavaScript para cálculo de Potência e Energia:

```
// Normaliza a intensidade: converte valor DMX (0-255) para percentagem
var val = parseFloat(Value);
if (val > 1) val = val / 255;

// Calcula o Consumo Instantâneo de Potência (Potência em Watts)
// P = Intensidade Normalizada * Potência Máxima
var P = val * parseFloat(MaxPower);

// Calcula a Energia (Dur é o tempo entre cues, em segundos)
var dur = parseFloat(betweenCuesTime);
var E_J = P * dur; // Energia em Joules
var E_Wh = E_J / 3600; // Energia em Watt-hora (Wh)
```

```
// Arredonda para duas casas decimais e define os campos de saída
PowerUsed = Math.round(P * 100) / 100;
Energy_J   = Math.round(E_J * 100) / 100;
Energy_Wh  = Math.round(E_Wh * 100) / 100;
```

- **Saída:** Cria os ficheiros `fixture_consumption.csv` e `dimmer_consumption.csv`.

4.4 4. Graph_Creation.ktr (Geração de Gráficos)

Esta transformação é responsável por processar os dados de consumo agregados e acionar a geração de gráficos.

- **Passo Chave:** *Execute a process.*
- **Lógica:** O comando de execução do Python é montado, sendo o caminho do script Python passado como um **argumento fixo** para o processo.

4.4.1 Configuração de Caminho Absoluto do Script Python

Apesar dos esforços para usar variáveis de ambiente do Kettle (`$Internal.Entry.Current.Directory` ou `$PYTHON_DIR`) para garantir a portabilidade do caminho do script, o passo *Execute a process* do Kettle, em alguns ambientes, requer que o caminho do script *Python* seja definido como um **caminho absoluto e fixo** para ser executado corretamente.

Esta necessidade de caminho absoluto compromete a portabilidade imediata do projeto, sendo este um ponto crítico de configuração.

- **Nodo a Editar:** *Add Arguments* (dentro de *Graph_Creation.ktr*).
- **Campo a Atualizar:** O campo `file-path` deve ser editado, substituindo o caminho atual (e.g., `H:\LESI\ISI\tp01-27967\scripts\graph_generator.py`) pelo caminho absoluto do `graph_generator.py` no ambiente de execução.

O valor configurado atualmente no passo *Add Arguments* é:

```
H:\LESI\ISI\tp01-27967\scripts\graph_generator.py
```

Este valor deve ser substituído pelo caminho local.

4.5 5. SendEmail.ktr (Envio de Relatório)

Esta transformação trata do envio dos resultados por e-mail.

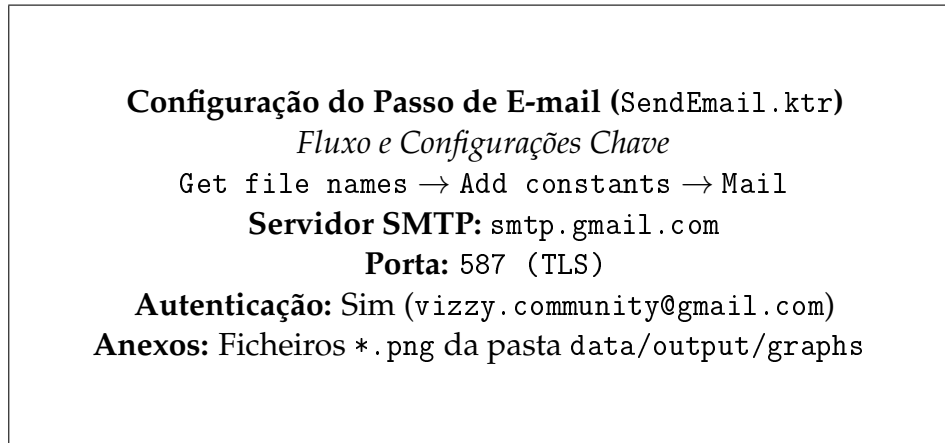


Figura 4: Esquema da transformação de envio de e-mail, que recolhe os gráficos e os distribui de forma segura.

- **Função:** Recolhe os gráficos (*.png) gerados e envia-os como anexos.
- **Segurança:** Requer a utilização de autenticação (TLS ou SSL) e credenciais válidas do servidor de e-mail (por exemplo, uma *app password* para contas Gmail ou Office 365).

5 Fluxo de Visualização Node-RED

O Node-RED é utilizado para fornecer uma visualização imediata dos dados de consumo gerados pelo Kettle, através de um **dashboard** interativo. O fluxo (node-red-graph-generator.json) é composto por uma sequência de nós que lê, processa e apresenta os dados.

5.1 Configuração de Caminhos e Ficheiros

O fluxo do Node-RED utiliza nós `file in` e `inject` para ler os ficheiros CSV gerados pelo Kettle. Estes nós foram configurados com caminhos fixos (**absolutos**), o que requer uma **configuração manual** por parte do utilizador.

- **Caminho Base:** O caminho absoluto atual é `H:\LESI\ISI\tp01-27967\data\output\`.
- **Caminho a Substituir:** O utilizador deve substituir a porção do caminho que aponta para a pasta do projeto (`H:\LESI\ISI\tp01-27967`) pelo caminho local onde o projeto está guardado.

Os nós que requerem esta atualização são os nós `inject` (`Read Fixture CSV` e `Read Dimmer CSV`) e os nós `file` de saída. Por exemplo, o caminho:

`H:\LESI\ISI\tp01-27967\data\output\fixture_consumption.csv`

deve ter a parte `H:\LESI\ISI\tp01-27967` substituída pelo *path* local do novo utilizador.

5.2 Visão Geral do Fluxo Node-RED

O fluxo Node-RED desempenha as seguintes tarefas principais:

1. **Leitura e Injeção:** Utiliza nós `inject` para carregar manualmente os ficheiros `fixture_consumption.csv` e `dimmer_consumption.csv`.
2. **Parsing e Conversão:** Os dados CSV lidos são convertidos em **arrays** de objetos JavaScript (nós `function`).
3. **Junção e Achatamento (Flatten):** Os dados de **Fixtures** e **Dimmers** são unidos (nó `join`) e convertidos numa lista única (nó `Flatten Arrays`).
4. **Agregação e Transformação de Gráficos:** Os dados são agregados por *Layer* e *FixtureType*, e formatados para diferentes tipos de gráficos (Barras, Circular) e Tabelas.
5. **Cálculo Total:** É calculada a *Total Power Usage* e *Total Energy Usage* para apresentar em indicadores (*gauges*).
6. **Visualização e Saída de Dados:** Os dados são apresentados no **dashboard** (nós `ui_chart`, `ui_table`, `ui_gauge`) e, em paralelo, os dados agregados para os gráficos são exportados como JSON (nós `file`) para uso futuro pelo *script Python* (e.g., `layer-graph-data.json`).
7. **Orquestração HTTP:** O fluxo inclui um nó `http in (/trigger)` para ser acionado remotamente pelo Kettle, iniciando o pipeline de processamento e visualização no Node-RED.

5.3 Esquema de Agregação de Dados (Node-RED)

Os nós function para agregação (e.g., Aggregate Power by Layer) são cruciais, pois transformam o fluxo de dados plano (por *Cue*/aparelho) numa estrutura sumarizada que é adequada para gráficos.

Snippet do JavaScript para Aggregate Power by Layer:

```
let data = {};  
msg.payload.forEach(row => {  
    let layer = row.Layer || 'Unknown';  
    // Soma o PowerUsed de todas as linhas por camada  
    data[layer] = (data[layer] || 0) + parseFloat(row.PowerUsed || 0);  
});  
  
msg.payload = [{  
    series: ["PowerUsed"],  
    data: [ Object.values(data) ],  
    labels: Object.keys(data)  
}];  
  
return msg;
```

5.4 Componentes de Visualização (Dashboard)

O **dashboard** é estruturado para apresentar os principais indicadores:

- **Gráficos de Barras e Circulares:** Apresentam a distribuição do consumo (Watts) por *Layer* e por *Fixture Type*.
- **Tabelas (ui_table):** Exibem os dados agregados em formato tabular, facilitando a consulta de valores específicos.
- **Indicadores (ui_gauge):** Mostram o consumo total de Potência (Watts) e Energia (Watt-hora) de forma visual.

6 Conclusão e Trabalhos Futuros

O projeto demonstrou com sucesso a aplicação de um pipeline de ETL utilizando o Pentaho Data Integration para processar dados heterogêneos (XML, CSV) provenientes de um sistema de controlo de iluminação profissional (grandMA2). O fluxo de trabalho alcançou o objetivo de calcular o

consumo de energia instantâneo e total por aparelho e por camada (Layer) de iluminação, culminando na geração automatizada de relatórios visuais (Python/Node-RED) e na distribuição por e-mail.

6.1 Limitações e Melhorias Futuras

A principal limitação atual reside na precisão temporal do cálculo de consumo e na dependência de caminhos de ficheiros fixos:

- **Tempo de Cue Simulada:** O tempo de duração entre *cues* é atualmente um valor aleatório (simulado), o que torna o cálculo da energia (Wh) aproximado.
- **Falta de Portabilidade:** A necessidade de codificar caminhos absolutos (no Kettle e Node-RED) compromete a portabilidade "plug-and-play" do projeto.

Para endereçar estas limitações, o principal trabalho futuro sugerido é:

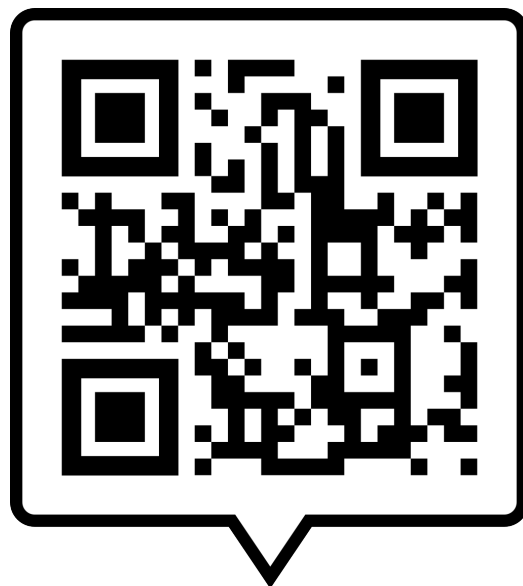
- **Implementação de Logger em LUA/Python na grandMA2:** Desenvolver um **plugin** ou **macro** dentro da consola grandMA2 que corra em tempo real durante o espetáculo. Este *plugin* faria um *log* do *timestamp* da mesa (utilizando o relógio interno da grandMA2) cada vez que é dado um "Go" (mudança de *cue*). A exportação deste *log* permitiria ao Kettle utilizar os **tempos de transição reais**, substituindo o valor aleatório simulado e garantindo a máxima precisão no cálculo de consumo energético total.

7 Repositório Git

O repositório completo do projeto pode ser encontrado no seguinte link.

8 Vídeo de demonstração

O vídeo de demonstração pode ser acessado através do seguinte QR Code.



SCAN ME

Figura 5: QR Code para vídeo de demonstração.